

Database and Storage Design

1. Database status

PixelGrid stores runtime state in microcontroller memory while a game is running. Optional score submission sends a score to an external API, but the repository does not include the external backend or its database.

2. Storage architecture

2.1 Runtime-only storage

The firmware uses volatile in-memory data structures:

- Tetris board, piece, score, level, and timing state in `TetrisGame`.
- Breakout bricks, paddle, ball, score, timers, and game-over state in `Games/Breakout/Game.cpp` and `Game.h`.
- Pixel buffers and LED conversion tables in `Pixel_Grid`.
- LCD digit state in `LCD_Digit` and `LCD_Panel`.
- Input debounce state in Tetris and Breakout input modules.
- Host runtime buffers in `Games/Tetris/HostRuntime.cpp`.

When the device resets or power is lost, this state is lost.

2.2 External score storage

`Games/Tetris/NetSubmit.h` submits a signed score to `/api/codes`. The external service may persist submissions or generated codes, but that storage is outside this repository and cannot be documented from local evidence.

2.3 Configuration storage

Configuration is compile-time source/header data rather than a database:

- Pin and timing constants are compiled from `Pins.h` files.
- Network/API values are expected from `NetConfig.h`, which is not present.
- Arduino library metadata is stored in `library.properties`.
- CI configuration is stored in YAML under `.github/workflows`.

3. In-memory entity descriptions

3.1 Tetris entities

Entity	Fields	Source
Board	<code>board[PLAY_H][W]</code> , where 0 is empty and 1..7 are occupied piece types.	<code>Games/Tetris/Game.h</code>
Piece	<code>type</code> , <code>rot</code> .	<code>Games/Tetris/Game.h</code>

Entity	Fields	Source
Game status	<code>gameOver</code> , <code>score</code> , <code>totalLinesCleared</code> , <code>level</code> , <code>fallDelayMs</code> , <code>lastScoreSpeedStep</code> , <code>tFall</code> .	<code>Games/Tetris/Game.h</code>
Hold state	<code>holdType</code> , <code>holdLocked</code> .	<code>Games/Tetris/Game.h</code>
Colours	<code>PIECE_COLORS</code> , <code>PREVIEW_BG</code> , <code>PLAY_BG</code> , <code>GHOST_COLOR</code> .	<code>Games/Tetris/Game.h</code> , <code>Games/Tetris/Render.h</code>

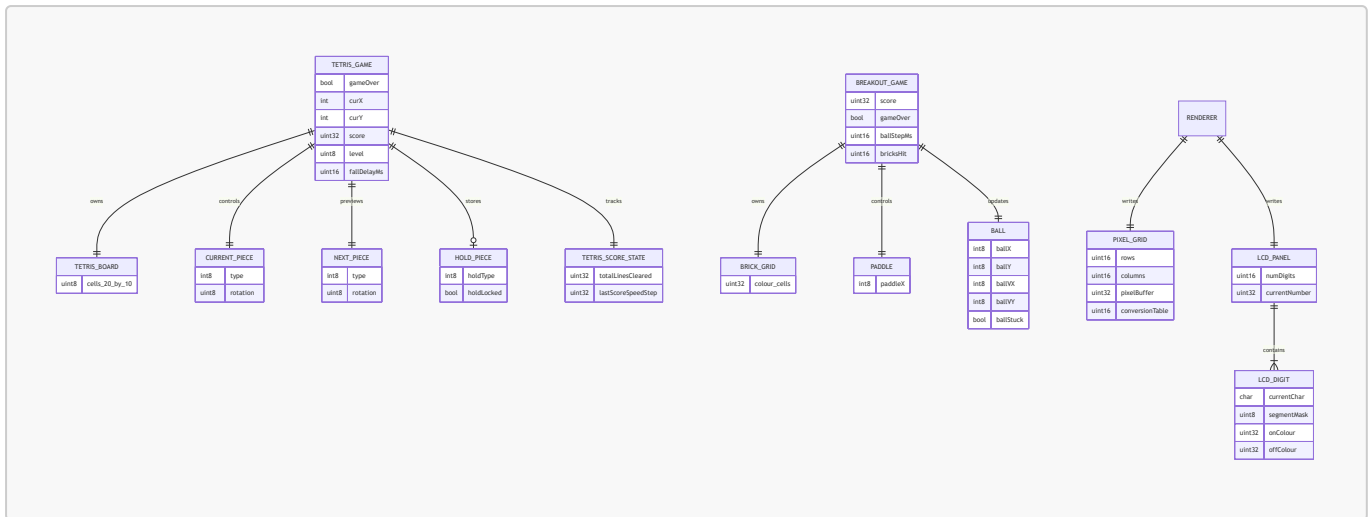
3.2 Breakout entities

Entity	Fields	Source
Brick grid	<code>bricksGrid[BRICK_H][W]</code> colour values.	<code>Games/Breakout/Game.cpp</code> , <code>Game.h</code>
Paddle	<code>paddleX</code> , configured paddle width and row.	<code>Games/Breakout/Game.cpp</code> , <code>Pins.h</code>
Ball	<code>ballX</code> , <code>ballY</code> , <code>ballVX</code> , <code>ballVY</code> , <code>ballStuck</code> , <code>ballStepMs</code> .	<code>Games/Breakout/Game.cpp</code> , <code>Game.h</code>
Game status	<code>score</code> , <code>gameOver</code> , <code>bricksHit</code> , timing variables.	<code>Games/Breakout/Game.cpp</code> , <code>Game.h</code>

3.3 Display entities

Entity	Fields	Source
Pixel grid	Number of rows/columns, pixel buffer, conversion table, start index, NeoPixel strip pointer.	<code>libraries/PixelGridcore/src/Pixel_Grid.h</code>
LCD digit	Current digit/character, segment mask, colours, start index.	<code>libraries/PixelGridcore/src/LCD_Digit.h</code>
LCD panel	Array of six digits, current number, character array, strip pointer.	<code>libraries/PixelGridcore/src/LCD_Panel.h</code>

4. ERD-style model for runtime state



5. Constraints and indexes

Runtime constraints are enforced procedurally:

- Board dimensions are enforced by checks in Tetris placement validation.
- Breakout paddle and ball positions are constrained by game functions and pin/dimension constants.
- Pixel grid writes depend on logical row/column conversion through `Pixel_Grid`.
- Host packets enforce expected payload lengths before state updates.
- Score submission requires a timestamp and HMAC signature before HTTP POST.